Nivel 1

Descarga los archivos CSV, estudiales y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:

1) Crear BBDD sprint4:

```
drop database sprint4;
create database sprint4;
```

Output

Action Output

#	Time	Action	Message
1	11:34:45	drop database sprint4	7 row(s) affected
2	11:34:45	create database sprint4	1 row(s) affected

2) Creo tabla users (para simplificar american_users y european_users), credit_card, companies y transactions:

```
create table users (
    id int primary key,
    name varchar(20),
    surname varchar(20),
    phone varchar(40),
    email varchar(50),
    birth_date varchar(20),
    country varchar(20),
    city varchar(40),
    postal_code varchar(40),
    address varchar(50),
    region enum('America','Europe')
);

create table credit_card (
    id varchar(20) primary key,
    user_id int,
    iban varchar(50),
    pan varchar(50),
    pin varchar(5),
    cvv varchar(5),
    track1 varchar(100)
```

Output

Action Output

#	Time	Action	Message
1	11:38:40	use sprint4	0 row(s) affected
2	11:38:40	create table users (id int primary key, name varchar(20), surname varchar(20), phone varchar(40), email varchar(50), birth_date varchar(20), ...	0 row(s) affected
3	11:38:40	create table credit_card (id varchar(20) primary key, user_id int, iban varchar(50), pan varchar(50), pin varchar(5), cvv varchar(5), track1 ...	0 row(s) affected
4	11:38:40	create table companies (company_id varchar(50) primary key, company_name varchar(50), phone varchar(40), email varchar(50), country var...	0 row(s) affected
5	11:38:40	create table transactions (id varchar(50) primary key, card_id varchar(20), business_id varchar(50), timestamp timestamp, amount decimal(10, 2), ...	0 row(s) affected

3) Cargar las tablas con los archivos csv y agregar datos de región en american_users y european_users.

```
load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/S04_datos/american_users.csv'
into table users
fields terminated by ','
enclosed by '"'
ignore 1 rows
(id, name, surname, phone, email, birth_date, country, city, postal_code, address)
set region = "America";

load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/S04_datos/european_users.csv'
into table users
fields terminated by ','
enclosed by '"'
ignore 1 rows
(id, name, surname, phone, email, birth_date, country, city, postal_code, address)
set region = "Europe";

load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/S04_datos/credit_cards.csv'
into table credit_card
```

Output

Action Output

#	Time	Action	Message
1	11:40:12	load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/S04_datos/american_users.csv' into table users fields terminated by ',' enclosed b...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0
2	11:40:12	load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/S04_datos/european_users.csv' into table users fields terminated by ',' enclosed b...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0
3	11:40:12	load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/S04_datos/credit_cards.csv' into table credit_card fields terminated by ',' enclosed...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0
4	11:40:12	load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/S04_datos/companies.csv' into table companies fields terminated by ',' enclosed b...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0
5	11:40:12	load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/S04_datos/transactions.csv' into table transactions fields terminated by ',' ignore 1 ...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0

4) Relacionar tablas users con transactions:

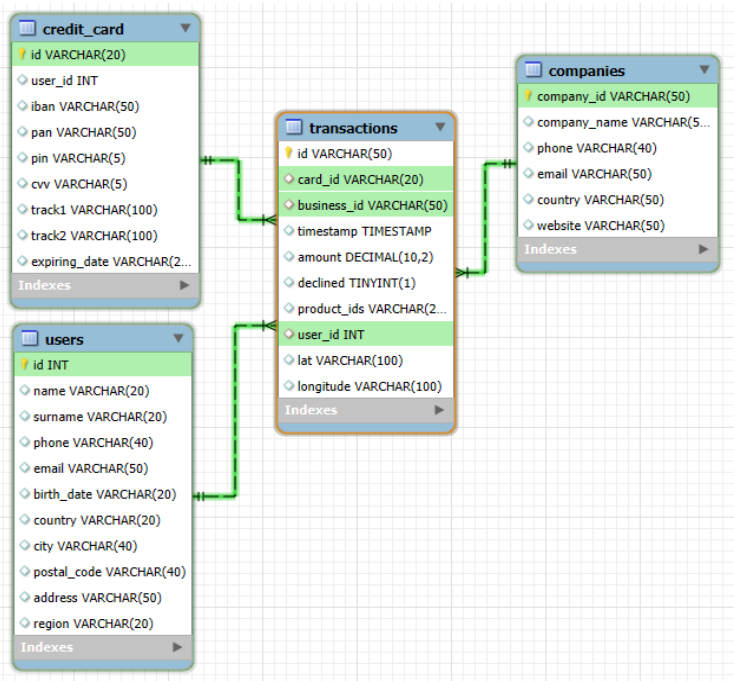
```
alter table transactions
add constraint users_transactions
foreign key (user_id)
references users(id);
```

Output

Action Output

#	Time	Action
1	20:34:43	alter table transactions add constraint users_transactions foreign key (user_id) references users(id)

5) Esquema estrella armado:



Ejercicio 1

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

```
select u.id,
       u.name as name,
       u.surname as surname,
       count(t.id) as transaction_count
from users as u
join transactions as t on t.user_id = u.id
where u.id in (select t.user_id
               from transactions as t
               where t.declined = 0
               group by t.user_id
               having count(t.id) > 80)
group by u.id, u.name, u.surname;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	name	surname	transaction_count
185	Molly	Gilliam	110
289	Dxwgi	Hwcru	94
318	Bnyr	Astuw	91

Result 21

Output

Action Output

#	Time	Action	Message
1	11:44:06	select u.id, u.name as name, u.surname as surname, count(t.id) as transaction_count from users as u join transactions as t on t.user_id = u.id...	3 row(s) returned

Ejercicio 2

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

```
select card.iban,
       avg(t.amount) as avg_amount,
       c.company_name as company
from transactions as t
join companies as c on c.company_id = t.business_id
join credit_card as card on card.id = t.card_id
where c.company_name = "Donec Ltd"
group by card.iban
order by avg_amount desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban	avg_amount	company
XX3830178139196201993663 52	680.690000	Donec Ltd
XX6377063573975703949739 13	680.010000	Donec Ltd
XX9713939714652922023122 59	645.460000	Donec Ltd
XX1718471169288923759693 07	628.890000	Donec Ltd
XX2254246388185424062235 75	608.680000	Donec Ltd
XX7488907290571957117660 71	607.290000	Donec Ltd
TN9614563570667381893122	605.410000	Donec Ltd

Result 22

Output

Action Output

#	Time	Action	Message
1	11:44:44	select card.iban, avg(t.amount) as avg_amount, c.company_name as company from transactions as t join companies as c on c.company_id =...	371 row(s) returned

Nivel 2

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas entonces es inactivo, si al menos una no es rechazada entonces es activo .

```sql
create table card_status (
    card_id varchar(20) primary key,
    status varchar(20)
);

insert into card_status (card_id, status)
select card_id,
       case when sum(declined) = 3 then 'Inactive'
       else 'Active'
       end as status
from (select card_id,
             declined,
             row_number() over(
                partition by card_id
                order by date(timestamp) desc
             ) as sort_order
      from transactions) t
where sort_order <= 3
group by card_id;

select * from card_status;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

card_id	status
CcS-4857	Active
CcS-4858	Active
CcS-4859	Active
CcS-4860	Active
CcS-4861	Active

card_status 23

Output

Action Output

#	Time	Action	Message
1	11:46:26	create table card_status (card_id varchar(20) primary key, status varchar(20))	0 row(s) affected
2	11:46:26	insert into card_status (card_id, status) select card_id, case when sum(declined) = 3 then 'Inactive' else 'Active' end as status from (select card_id,...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
3	11:46:26	select * from card_status LIMIT 0, 1000	1000 row(s) returned

Partiendo de esta tabla responde:

Ejercicio 1

¿Cuántas tarjetas están activas?

```sql
select count(status) as active_count
from card_status
where status = "Active";
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

active_count
4995

Result 24

Output

Action Output

#	Time	Action	Message
1	11:47:41	select count(status) as active_count from card_status where status = "Active" LIMIT 0, 1000	1 row(s) returned

Nivel 3

Crea una tabla con la que podamos unir los datos del nuevo archivo products.csv con la base de datos creada, teniendo en cuenta que desde transaction tienes product_ids.

1) Crear tabla **products**:

```sql
create table products (
    id int primary key,
    product_name varchar(40),
    price varchar(20),
    colour varchar(40),
    weight varchar(40),
    warehouse_id varchar(20)
);
```

Output

Action Output

#	Time	Action	Message
1	11:49:18	create table products (id int primary key, product_name varchar(40), price varchar(20), colour varchar(40), weight varchar(40), warehouse_i...	0 row(s) affected

2) Cargar csv:

```sql
load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/products.csv'
into table products
fields terminated by ','
enclosed by '"'
ignore 1 rows;
```

Output

Action Output

#	Time	Action
1	10:29:45	load data infile 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/products.csv' into table products fields terminated by ',' enclosed by '"' ignore 1 rows

3) Crear tabla puente **product_transactions** con clave primaria compuesta para permitir múltiples product_id por transaction_id y múltiples transaction_id por product_id.

```sql
create table product_transactions (
    transaction_id varchar(50),
    product_id int,
    primary key (transaction_id, product_id)
    );
```

Output

Action Output

#	Time	Action	Message
1	11:50:50	create table product_transactions (transaction_id varchar(50), product_id int, primary key (transaction_id, product_id))	0 row(s) affected

4) Desde **transactions** insertamos los registros transaction_id y product_id los obtendremos utilizando JSON para separarlos en columnas distintas.

```
insert into product_transactions (transaction_id, product_id)
select
    t.id,
    j.value as product_id
from transactions t
join json_table(                    -- interpreta el texto como json
    concat('[', t.product_ids, ']'), -- esto me deja los arrays asi [1,2,3] donde las comas son parte de la sintaxis oficial de json
    '$[*]' columns (                -- esto me dice: devuelveme cada elemento del array en filas separadas ($ -> el documento completo, [*] -> cada elemento del array)
        value int path '$'          -- cada elemento se convierte directamente a entero
    )
) AS j;
```

Output

Action Output

#	Time	Action	Message
1	11:51:46	insert into product_transactions (transaction_id, product_id) select t.id, j.value as product_id from transactions t join json_table(-- interp...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0

5) Relacionar tablas **product_transactions** con **products** y **transations**.

```
alter table product_transactions
add constraint pt_transactions
foreign key (transaction_id)
references transactions(id);
```

Output

Action Output

#	Time	Action
1	11:04:44	alter table product_transactions add constraint pt_product foreign key (product_id) references products(id)
2	11:04:47	alter table product_transactions add constraint pt_transactions foreign key (transaction_id) references transactions(id)

Genera la siguiente consulta:

Ejercicio 1

Necesitamos conocer el número de veces que se ha vendido cada producto.

```
select product_id, count(product_id) as times_sold
from product_transactions
group by product_id;
```

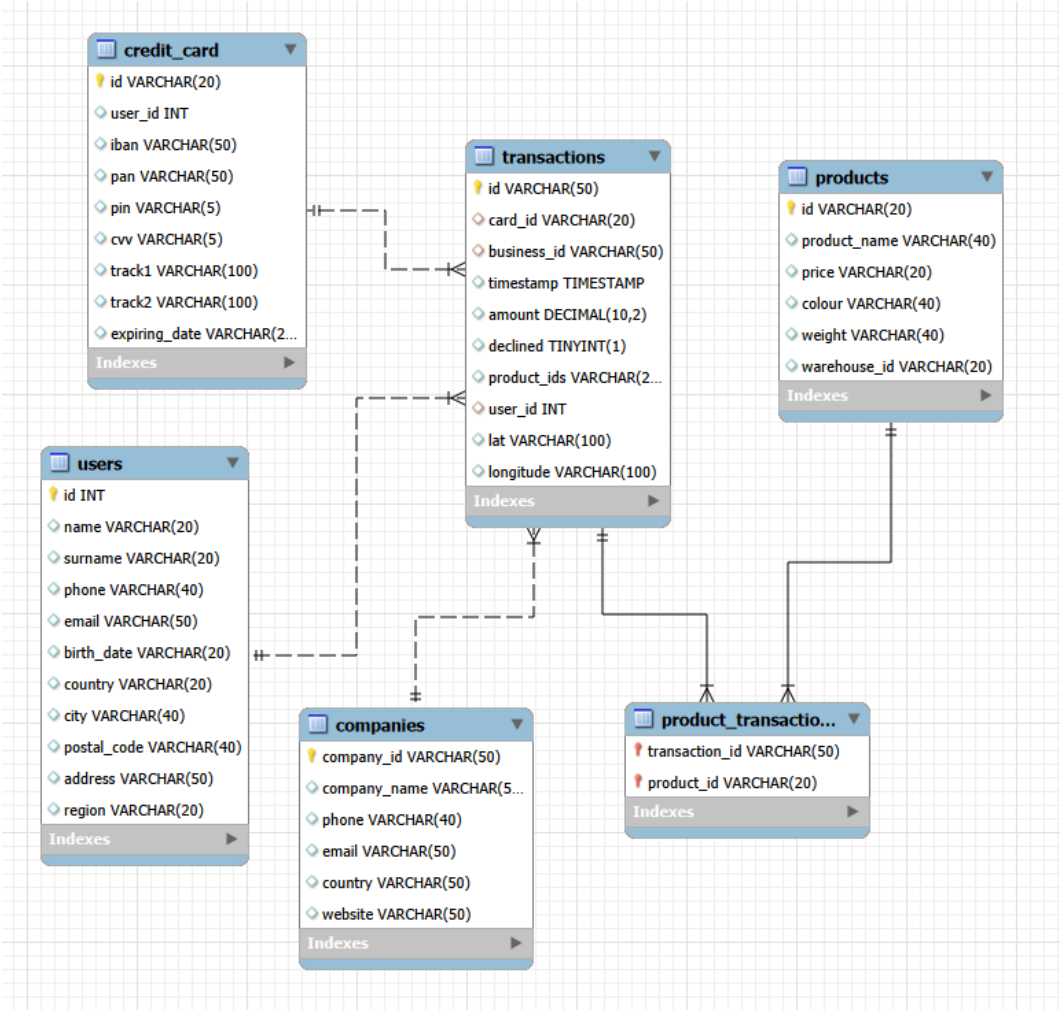
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	times_sold
1	2468
2	2573
3	2528
4	2584
5	2548
6	2492

Result 25

Output

Action Output

#	Time	Action	Message
1	11:54:01	select product_id, count(product_id) as times_sold from product_transactions group by product_id LIMIT 0, 1000	100 row(s) returned

Esquema Final